AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_HWSW_VGA_IP Version : 1	

HW/SW IP KOMPONENT-
PROTOTYP AV VGA IP KOMPONENT

Darwin Rengifo

2019-04-19

Sammanfattning:
Denna rapport beskriver en förstudie och planering för VGA IP projektet med användningen av IP komponenter och Qsys-vektyget för att integrerar hårdvaran och mjukvaran interface genom läsningen och skrivningen på Avalon bus. Det visar några texter på VGA-skärmen för att validerats IP komponents funktionalitet. Denna rapport innehåller aktivitets- och tidplan, projektkostnad, kravspecifikations, projektarkitektur och beskrivning av utvecklade komponenter.

Innehåll

1. INLEDNING 3
2. KRAVSPECIFIKATION FRÅN KUND 4
3. PROJEKPLAN 7
4. HÅRDVARAN ARKITEKTUR FÖR INGÅENDE KOMPONENTER. 8
4.1 Komponenter 8
4.1.1 Vga_ip 8
4.1.2 Vga_hw_ip 8
4.1.2.1 Vga_component 8
4.1.2.1.1 Vga_sync 8
4.1.2.1.2 Dp_ram 8
4.1.3 Timer_hw_IP 9
4.1.3.1 Timer_top 9
4.1.4 PLL 9
4.1.5 System ID 9
4.1.6 On-Chip RAM 9
4.1.7 JTAG UART 9
4.1.8 SDRAM Controller 9
5. KONSTRUKTIONS BESKRIVNING 10
5.1 Systemarkitektur 10
5.2 Delsystem 11
5.2.1 Toppnivån 11
5.2.1.1 Systemarkitektur 11
5.2.2 Vga IP – komponent 13
5.2.2.1 Systemarkitektur 13
5.2.3 Vga kontroller - komponent 14
5.2.3.1 Systemarkitektur 14
5.2.4 Ram kontroller - komponent 16
5.2.4.1 Systemarkitektur 16
5.2.5 Synkronisering vga - komponent 17
5.3 I/O pinnar 18

6. VALIDERING KONSTRUKTIONEN 19
6.1 Testprotokoll för validering 19
7. MJUKVARAN ARKITEKTUR 20
7.1 Minneskarta 20
7.2 Drivrutiner 20
7.2.1 VGA färger 20
7.2.2 VGA-drivrutiner 20
7.3 C koden 21

1. INLEDNING

Denna rapport beskriver som bygger en avståndsmätare med ultraljudsensor och alla IP komponenterna som kommer att bygga för att se resultat av visualisering på sjusegmentsdisplayerna som finns på utvecklingskortet och VGA-skärmen som ska det visar färger röd, grön, blå och vit för olika avstånd. Det kommer att validerats på ett lämpligt FPGA-kort och VGA-skärmen.

2. KRAVSPECIFIKATION FRÅN KUND

Tabell 1. Kravspecifikation från kund

Krav	Beskrivning	Utfört
		Ja/nej
001	Kundprojektets arbetsnamn är VGA_IP	
002	Skapa en IP komponent med VGA-komponenten och återanvänd tidigare VHDL- och C-projekt.	
003	Krav på konstruktionskoden: 1)"filhuvud" i C-filen, 2) Namn på variabler m.m. ska vara tydliga, 3) Rikligt med kommentarer i koden, 4) Frivilligt: Kontroll av gränsvärden ska göras, t.ex. att värdet inte ligger utanför skärmen. Om detta inträffar ska ett felmeddelande skrivas ut till systemterminalen (Nios II "console").	
Funktionskrav		
004	Följande drivrutiner sparas under HAL och ska vara uppdaterade i VGA_IP_SW_sw.tcl filen). Drivrutinerna kan kopieras från tidigare utfört projekt i C-kursen eller från ett äldre projekt. Funktion: print_pix(unsigned int x,unsigned int y,unsigned int rgb); Funktionsbeskrivning: Skriver en pixel med färgen rgb på koordinaten (x, y). Alternativ lösning för print_pix; om funktionen behöver snabbas upp, kan den istället göras om till ett macro.	

	Funktion: print_hline(unsigned int x_start, unsigned int y_start, unsigned int len, unsigned int RGB); Funktionsbeskrivning: Skriver en horisontell linje med färgen rgb och med längden len vilken startar på koordinaten (x_start, y_start). Funktion: print_vline(unsigned int x_start, unsigned int y_start, unsigned int len, unsigned int RGB); Funktionsbeskrivning: Skriver en vertikal linje med färgen rgb och med längden len vilken startar på koordinaten (x_start, y_start). Funktion: print_char(unsigned int x,unsigned int y,unsigned int rgb,unsigned int BG_RGB,char Character); Funktionsbeskrivning: Skriver tecknet "Character" med färgen rgb och med bakgrundsfärgen "BG_RGB" på koordinaten (x, y). Lägg till de ASCI som behövs för att skriva det som krävs. Det går att bestämma hur tecknen ska visas på skärmen, se bilaga, äldre projekt och på webben. Funktion: read_pixel_ram_int(unsigned int x_start, unsigned int y_start); return: unsigned pixel_data (obs! enbart de sista tre bitarna är intressanta). Funktionsbeskrivning: Läser pixel_data från pixel RAM (3 bitar, RGB) från adress x och y (beräknat från x och y). Funktion: clear_screen (int rgb); Funktionsbeskrivning: Denna funktion rensar skärmen genom att RGB-värdet "000"(svart) skrivs till alla pixlar på skärmen. Funktion: print_circle(unsigned int radie, unsigned int x_centrum, unsigned int y_centrum, unsigned int rgb); Funktionsbeskrivning: Skriver en cirkel med radien "radie" och färgen" rgb" på mittkoordinaten (x, y). Cirkeln ska fyllas med samma färg.	
Valideringskrav		
005	Testprotokollet kan vara ofullständigt, men ska innehålla minst två testfall (HW/SW design report).	
006	Skriv ditt namn på skärmen, fotografera skärmen och lägg in fotot som figur i rapporten (HW/SW design report).	
007	Rapporten ska också innehålla kostnaden för konsultuppdraget. Företaget konstukören är anställd på fakturerar 600 SEK/timme för projektet. Totalt antal timmar * 600 SEK utgör den totala kostnaden.	
Struktur på leveransen och dokumentationskrav		
008	Leverera följande struktur med zippade filer. • VGA_IP (top library and the sub library) o VGA_IP_HW_hw.tcl o VGA_IP_SW_sw.tcl o hdl (HW), - ….vhd (VHDL code)	

	○ HAL (SW), - Inc (library for h files) - src (library for c files) ○ doc (documentations), - VGA_datasheet - Software manual for the programmers - SW/HW Design Report (standard report) ○ Examples - Ett exempel system som skriver ut ditt namn på skärmen - SOF och sopcinfo filen - Arkiverade projektet - C filen (main) ○ Projects (HW) - VGA_IP (Quartus) - VGA_IP_HW (Quartus)	
Leveranskrav		
009	Leveransen ska ske till plattformen Itslearning. Leveransen ska vara en IP-komponent enligt strukturen från krav 004. Namnet på filen ska vara ”förnamn_efternamn_HWSW_task_3.zip”. Sista leveransdag se kursschema (för VG).	

3. PROJEKPLAN

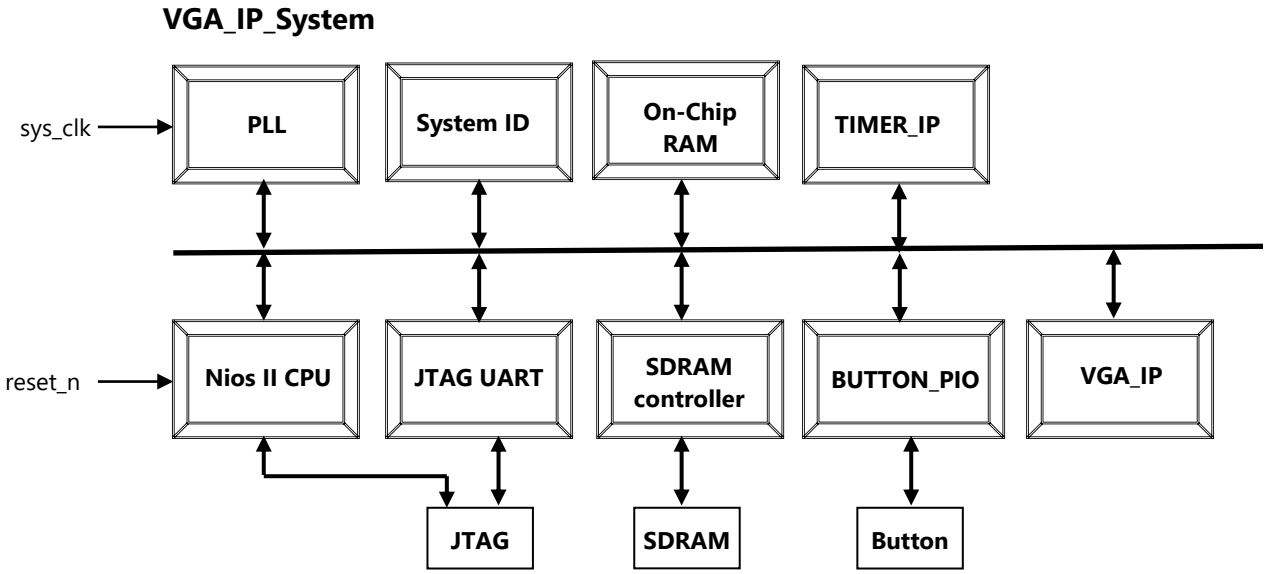
Aktivitets - och tidplan för ingenjörsjobbet kan ses i Tabell 2. Totalt uppskattas ingenjörsjobbet ta 28 timmar/600kr(timme), vilket ger en lönekostnad på 16 800 SEK.

Tabell 2. Aktivitets- och tidsplan för ingenjörsjobbet

Aktivitet	Veckor				
	46	47	48	Timmar(h)	Kostnad(kr)
Kravspec./förstudie	5			5	3 000
Testprotokoll/ Drivrutiner	1	1	1	3	1 800
Arkitekturbeskrivning	3	3		6	3 600
Programmering av visuellt gränssnitt	4	3	1	7	4 200
Verifiering		1	1	2	1 200
Validering			1	1	6 00
Slutrapport		2	2	4	2 400
Summa tid	13	10	6	28	16 800

4. HÅRDVARAN ARKITEKTUR FÖR INGÅENDE KOMPONENTER.

Systemarkitekturen som FPGA:n programmerats med är en enkel bussarkitektur för att läsa och skriver från IP komponenten med en mikroprocessor av typen Nios II som styr, se Figur 1.



Figur 1. Systemarkitektur, blockschema VGA_IP_System

4.1 Komponenter

I projektet ska flera komponenter utvecklas. En toppfilen komponent som instansierar qsys-interface, en för att skapa avalon bus interface och de andra för att skapar utgången interface av data till skärmen.

4.1.1 Vga_ip

Toppfilen som kommer att instansierar QSYS-interface.

4.1.2 Vga_hw_ip

Filen som skapar Avalon bus interface av läsning/skrivning även instansierar komponenten som skickar utgången signaler till skärmen.

4.1.2.1 Vga_component

Den komponenten skickar utgången signaler till skärmen.

4.1.2.1.1 Vga_sync

Denna VGA drivrutin skapar bilden av 640x480, men använder ett RAM-minne av 320x24

4.1.2.1.2 Dp_ram

Det är en "Dual Port Ram" med dubbel klock för att kan läsa och skriver information på minnet. Skrivning process på RAM minne är på 50mhz och läsning process från RAM minne är på 25mhz.

4.1.3 Timer_hw_IP

Filen som skapar Avalon bus interface av läsning/skrivning även instansierar komponent som använder för att ändrar hastighet frekvensen för att visualisering av data på skärmen.

4.1.3.1 Timer_top

Kontrollerar sekvensen av reset-stop-start-read på hårdvaran.

4.1.4 PLL

En PLL finns tillgänglig i systemet för att distribuera varianter av systemklockan på 50 MHz till delar av systemet.

4.1.5 System ID

En komponent som innehåller information om hårdvaruversion.

4.1.6 On-Chip RAM

Intern styrenhet för SDRAM-minnet.

4.1.7 JTAG UART

Kommunikationsgränssnitt mellan USB Blaster och data/instruktionsbus. Används för kommunikation mellan PC och CPU för programmering och debuggning.

4.1.8 SDRAM Controller

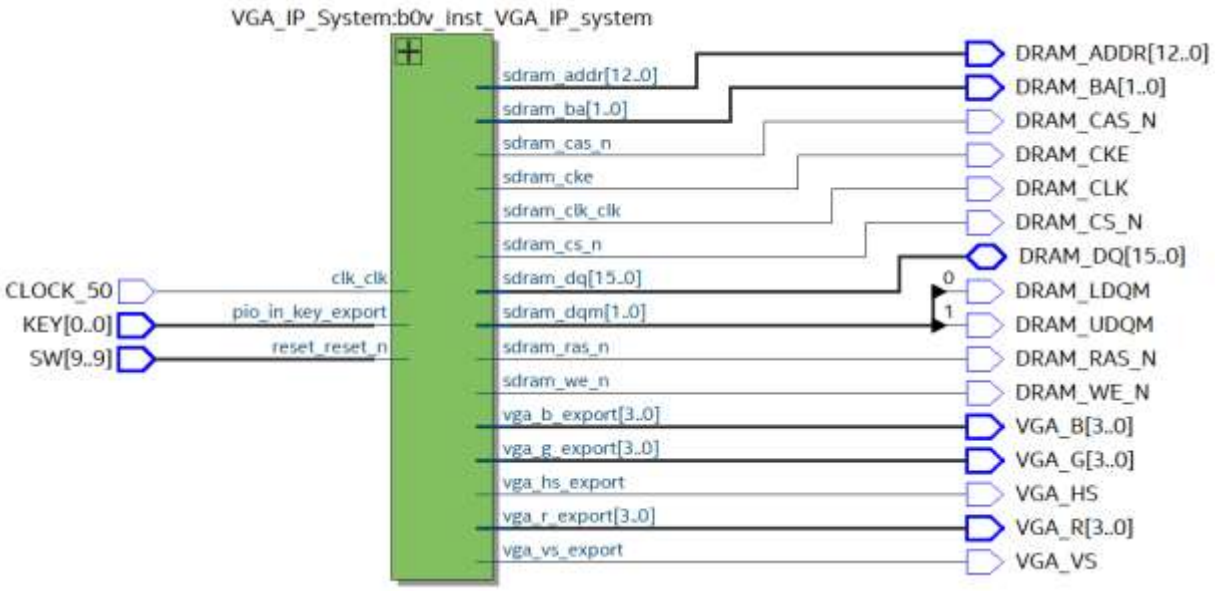
Extern styrenhet för SDRAM-minnet.

5. KONSTRUKTIONS BESKRIVNING

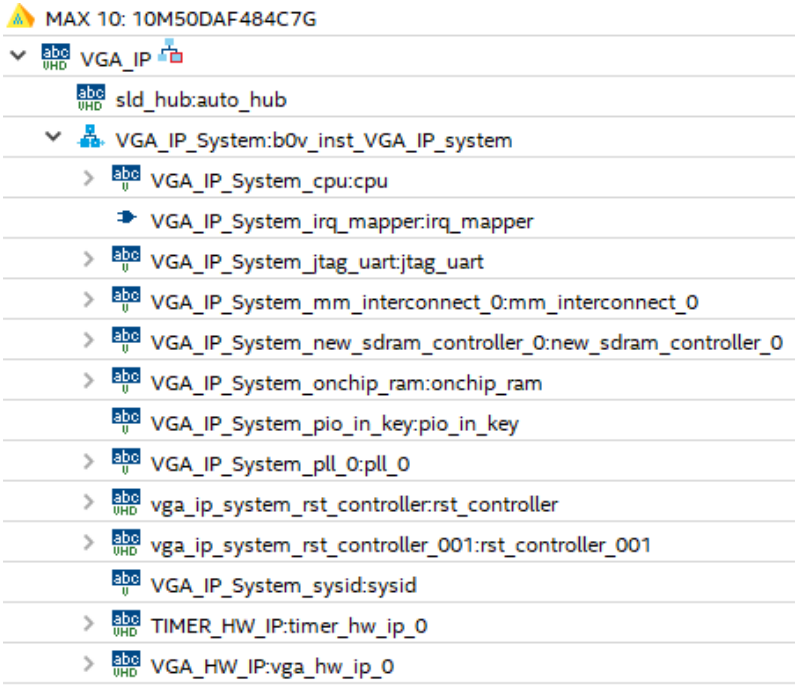
Programmet körs på kortet DE10-Lite är en robust hårdvara design plattform som byggde kring Altera MAX 10 FPGA med en Nios II-CPU som är anpassad för ingenjörsjobbet. DE10-Lite-kortet innehåller en 15-pin D-SUB-kontakt för VGA-utgång, VGA skärm för att visualisering resultaten, VGA-kabel för att skickar signalerna från DE10-Lite kortet till VGA skärm.

5.1 Systemarkitektur

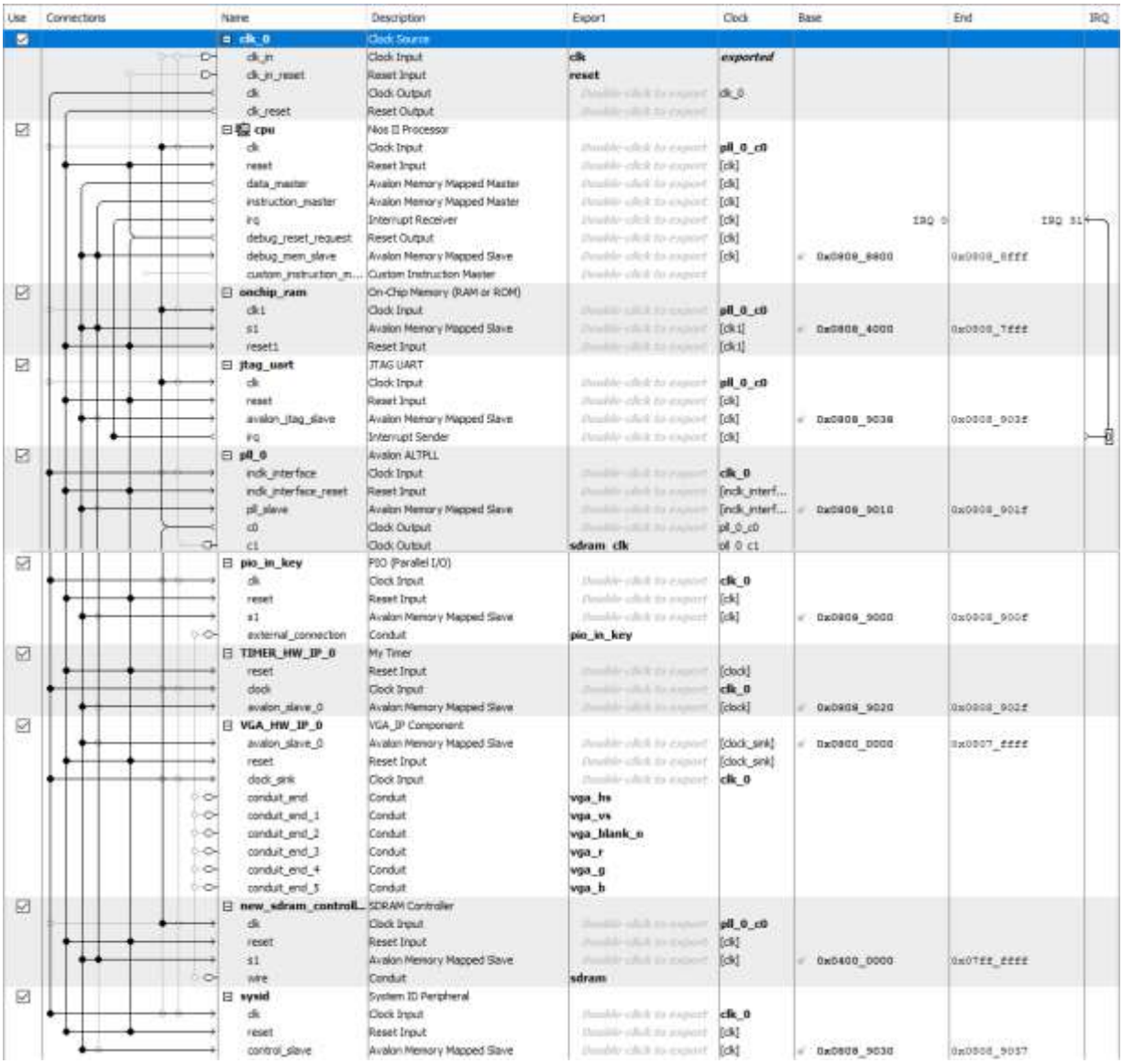
TOPPNIVÅN arkitektur visas i Figur 2, i figur 3 visar komponenthierarki och i figur 4 visar QSYS interface.



Figur 2. Systemarkitekturen för VGA_IP



Figur 3. Komponenthierarki för ultraljudsensor



Figur 4. QSYS interface för IP komponenter

5.2 Delsystem

Systemet innehålla följande komponenter:

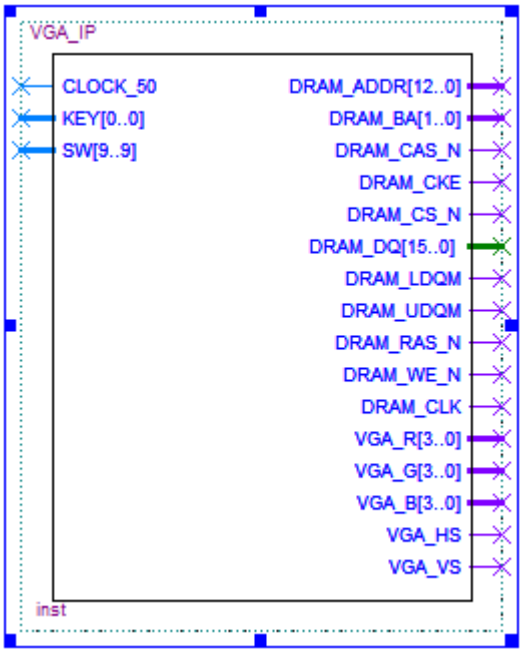
5.2.1 Toppnivån

Komponent namn: ***VGA_IP***

5.2.1.1 Systemarkitektur

Den toppnivån användas för att instansierar Qsys interface son innehålla hela IP komponenterna. Toppnivån får insignaler och skickar utsignaler till vga-skärmen för att rita texter på olika färger.

TOPPNIVÅN symbol visas i Figur 5 och i Tabell 3 visar In- och utsignalerna.



Figur 5. TOPPNIVÅN för VGA_IP

Tabell 3. In- och utsignalerna till TOPP filen

Signal	**Namn**	**Riktning**	**Typ**
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	SW	in	std_logic
Pio in_key	KEY	in	std_logic
External minne	SDRAM	out	std_logic, std_logic_vector
Horizontell synkronism	vga_hs	out	std_logic
Vertikal synkronism	vga_vs	out	std_logic
Röd signal	vga_r	out	std_logic_vector(3 downto 0)
Grön signal	vga_g	out	std_logic_vector(3 downto 0)
Blå signal	vga_b	out	std_logic_vector(3 downto 0)

Ingången signaler

CLOCK_50: 50Mhz input clock.

sw(reset_n): Reset av signalen. Active low.

echo: Pin som aktiverar sig för att börjar mäta avståndet. ”1” det börjar att räknar, ”0” slutar att räknar.

Key: skifta mellan presentations skärm och avstånd mäta skärmen.

Utgången signaler

SDRAM: Externa minne som innehålla data och instruktioner.

vga_hs: Horizontell synkronism. Den kontrollerar början och slutet på en rad och ser till att skärmen visar pixlarna mellan vänster och höger del av den synliga delen av bildskärmen.

vga_vs: Vertikal synkronism. Kontrollerar början och slutet på en ram(framme) och ser till att den visas mellan topp och nedre kanten och inom skärmen synliga ram, alla linjer som sminka.

vga_r: Definierar röd färg av signalen.

vga_g: Definierar grön färg av signalen.

vga_b: Definierar blå färg av signalen.

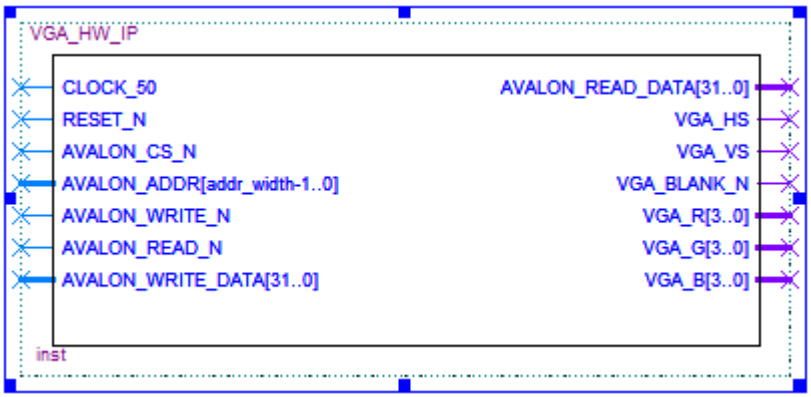
5.2.2 Vga IP – komponent

Komponent namn: **VGA_HW_IP**

5.2.2.1 Systemarkitektur

CPU'n genom denna ip_komponent läs eller skriv i busen Avalon för att skickar information till skärmen.

VGA IP-komponentens symbol visas i Figur 6 och i Tabell 4 visar In- och utsignalerna.



Figur 6. Avalon Bus interface för VGA_IP

Tabell 4. In- och utsignalerna till VGA_IP filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	RESET_N	in	std_logic
Chipselect	AVALON_CS_N	in	std_logic
Adress bus in	AVALON_ADDR	In	std_logic_vector(16 downto 0)
Write enable	AVALON_WRITE_N	In	std_logic
Read enable	AVALON_READ_N	In	std_logic
Data bus in	AVALON_WRITE_DATA	In	std_logic_vector(31 downto 0)
Data bus out	AVALON_READ_DATA	out	std_logic_vector(31 downto 0)
Horizontell synk	VGA_HS	out	std_logic

vertikal synk	VGA_HS	out	std_logic
Röd signal	VGA_R	out	std_logic_vector(3 downto 0)
Grön signal	VGA_G	out	std_logic_vector(3 downto 0)
Blå signal	VGA_B	out	std_logic_vector(3 downto 0)

Ingången signaler
CLOCK_50: 50Mhz input clock.
reset_n: Reset av signalen. Active low.
avalon_cs_nt: Active low.
avalon_addr: skriver eller läser på komponenten.
avalon_write_n: aktiverar skrivning på Avalon interface.
avalon_read_n: aktiverar läsning på Avalon interface
avalon_write_data: Data bus ingång som skriver till komponent.

Utgången signaler
avalon_read_data: Data bus utgång som läser från komponent.
vga_hs: Horizontell synkronism. Den kontrollerar början och slutet på en rad och ser till att skärmen visar pixlarna mellan vänster och höger del av den synliga delen av bildskärmen.
vga_vs: Vertikal synkronism. Kontrollerar början och slutet på en ram (framme) och ser till att den visas mellan topp och nedre kanten och inom skärmen synliga ram, alla linjer som sminka.
vga_r: Definierar röd färg av signalen.
vga_g: Definierar grön färg av signalen.
vga_b: Definierar blå färg av signalen.

5.2.3 Vga kontroller - komponent

Komponent namn: ***vga_component***
Instansnamn ***b0v_inst_vga_component***

5.2.3.1 Systemarkitektur

Den toppnivån användas för att får ingången signaler, instansierar de andra komponenter och skickar utgången signaler till skärmen.

VGA COMPONENT-komponentens symbol visas i Figur 7 och i Tabell 5 visar In- och utsignalerna.



Figur 7. VGA komponent för VGA_IP

Tabell 5. In- och utsignalerna till VGA_COMPONENT filen

Signal	**Namn**	**Riktning**	**Typ**
Klocksignal 50 MHz	clock_50	in	std_logic
Switch reset	reset_n	in	std_logic
adress	adress_vga_w	In	std_logic_vector(16 downto 0)
skriv data	data_vga_w	In	std_logic_vector(2 downto 0)
read-/write-signaler	write_enable	in	std_logic
cpu läs data	data_out	out	std_logic_vector(2 downto 0)
Horizontell synkronism	vga_hs	out	std_logic
vertikal synkronism	vga_vs	out	std_logic
sync pulses	vga_blank_n	out	std_logic
Röd signal	vga_r	out	std_logic_vector(3 downto 0)
Grön signal	vga_g	out	std_logic_vector(3 downto 0)
Blå signal	vga_b	out	std_logic_vector(3 downto 0)

Ingången signaler

cpu_clock: 50Mhz input clock.
reset_n: reset av signalen. Active low.
address_vga_w: skriver eller läser på komponenten.
data_vga_w: data bus ingång som skriver till minnet.
write_enable: aktiverar skrivning på ram minnet.

Utgången signaler

data_out: Data bus utgång som läser från minnet.
vga_hs: Horizontell synkronism. Den kontrollerar början och slutet på en rad och ser till att skärmen visar pixlarna mellan vänster och höger del av den synliga delen av bildskärmen.
vga_vs: Vertikal synkronism. Kontrollerar början och slutet på en ram(framme) och ser till att den visas mellan topp och nedre kanten och inom skärmen synliga ram, alla linjer som sminka.
vga_blank_n: Horisontell och vertikal blank.
vga_r: Definierar röd färg av signalen.
vga_g: Definierar grön färg av signalen.
vga_b: Definierar blå färg av signalen.

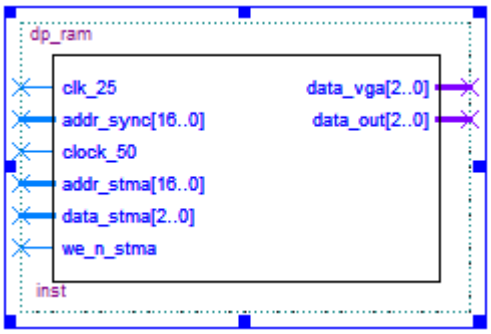
5.2.4 Ram kontroller - komponent

Komponent namn: ***dp_ram***
Instansnamn ***b2v_inst_dp_ram***

5.2.4.1 Systemarkitektur

Den komponenten används för att skriver och läsa data på minnet.

DP RAM-komponentens symbol visas i Figur 8 och i Tabell 6 visar In- och utsignalerna.



Figur 8. RAM kontroller för VGA_IP

Tabell 6. In- och utsignalerna till DP_RAM filen

Signal	Namn	Riktning	Typ
Klocksignal 25 MHz	clk_25	in	std_logic
Klocksignal 50 MHz	clock_50	in	std_logic
Upplösning	addr_sync	in	std_logic_vector(16 downto 0)
Testfall Adress	addr_stma	in	std_logic_vector(16 downto 0)
Testfall Data	data_stma	in	std_logic_vector(2 downto 0)
Styr skrivning	we_n_stma	in	std_logic
Data vga	data_vga	out	std_logic_vector(2 downto 0)
cpu läs data	data_out	out	std_logic_vector(2 downto 0)

Ingången signaler
clock_25: 25Mhz input clock.
clock_50: 50Mhz input clock.
reset_n: Reset av signalen. Active low.
addr_sync: 320 (pixel på en rad) * 240 (rader) upplösning.
addr_stma: skriver adress från tillståndmaskin.
data_stma: skriver data från tillståndmaskin.
we_n_stma: aktiverar skrivning på ram minnet.

Utgången signaler
data_vga: data utgång till vga skärmen
data_out: Data bus utgång som läser från minnet.

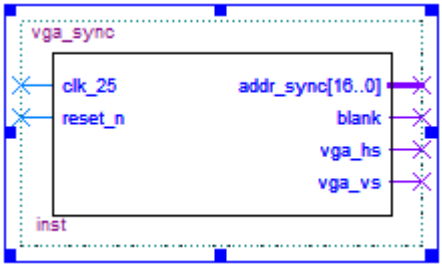
5.2.5 Synkronisering vga - komponent

Komponent namn: ***vga_sync***
Instansnamn ***b2v_inst_vga_sync***

5.2.5.1 Systemarkitektur

Den komponent användas för att räknar en skärm av 640 x 480 pixel för att kan aktivera horisontell, vertikal och mörk synkronisering även skapa adressen signal(addr_sync) som kommer att använda på grafikminnet. addr_sync är för 320 (pixel på en rad) * 240 (rader) upplösning.

VGA_SYNC komponent symbol visas i Figur 9 och tabell 7 visar In- och utsignalerna.



Figur 9. VGA synkronisering för VGA_IP

Tabell 7. In- och utsignalerna till VGA_SYNC filen

Signal	Namn	Riktning	Typ
Klocksignal 25 MHz	clk_25	in	std_logic
Switch reset	reset_n	in	std_logic
Upplösning.	addr_sync	out	std_logic_vector(16 downto 0)
Mörk skärm	blank	out	std_logic
Horizontell syn	vga_hs	out	std_logic
Vertikal sync	vga_vs	out	std_logic

Ingången signaler
clock_25: 25Mhz input clock.
clock_50: 50Mhz input clock.
reset_n: Reset av signalen. Active low.
addr_sync: 320 (pixel på en rad) * 240 (rader) upplösning.
addr_stma: skriver adress från tillståndmaskin.
data_stma: skriver data från tillståndmaskin.
we_n_stma: aktiverar skrivning på ram minnet.

Utgången signaler
data_vga: data utgång till vga skärmen
data_out: Data bus utgång som läser från minnet.

5.3 I/O pinar

Figur 10 visar pinar för DE10-Lite kortet.

Node Name	Direction
altera_reserved_tck	Input
altera_reserved_tdi	Input
altera_reserved_tdo	Output
altera_reserved_tms	Input
CLOCK_50	Input
DRAM_ADDR[12]	Output
DRAM_ADDR[11]	Output
DRAM_ADDR[10]	Output
DRAM_ADDR[9]	Output
DRAM_ADDR[8]	Output
DRAM_ADDR[7]	Output
DRAM_ADDR[6]	Output
DRAM_ADDR[5]	Output
DRAM_ADDR[4]	Output
DRAM_ADDR[3]	Output
DRAM_ADDR[2]	Output
DRAM_ADDR[1]	Output
DRAM_ADDR[0]	Output
DRAM_BA[1]	Output
DRAM_BA[0]	Output
DRAM_CAS_N	Output
DRAM_CKE	Output
DRAM_CLK	Output
DRAM_CS_N	Output
DRAM_DQ[15]	Bidir
DRAM_DQ[14]	Bidir
DRAM_DQ[13]	Bidir
DRAM_DQ[12]	Bidir
DRAM_DQ[11]	Bidir
DRAM_DQ[10]	Bidir
DRAM_DQ[9]	Bidir
DRAM_DQ[8]	Bidir

Node Name	Direction	Location
DRAM_DQ[7]	Bidir	PIN_V21
DRAM_DQ[6]	Bidir	PIN_W20
DRAM_DQ[5]	Bidir	PIN_W22
DRAM_DQ[4]	Bidir	PIN_Y22
DRAM_DQ[3]	Bidir	PIN_AA21
DRAM_DQ[2]	Bidir	PIN_AA22
DRAM_DQ[1]	Bidir	PIN_Y20
DRAM_DQ[0]	Bidir	PIN_Y21
DRAM_LDQM	Output	PIN_V22
DRAM_RAS_N	Output	PIN_U22
DRAM_UDQM	Output	PIN_J21
DRAM_WE_N	Output	PIN_V20
KEY[0]	Input	PIN_B8
SW[9]	Input	PIN_F15
VGA_B[3]	Output	PIN_N2
VGA_B[2]	Output	PIN_P4
VGA_B[1]	Output	PIN_T1
VGA_B[0]	Output	PIN_P1
VGA_G[3]	Output	PIN_R1
VGA_G[2]	Output	PIN_R2
VGA_G[1]	Output	PIN_T2
VGA_G[0]	Output	PIN_W1
VGA_HS	Output	PIN_N3
VGA_R[3]	Output	PIN_Y1
VGA_R[2]	Output	PIN_Y2
VGA_R[1]	Output	PIN_V1
VGA_R[0]	Output	PIN_AA1
VGA_VS	Output	PIN_N1

Figur 10. Pinnar för DE10-Lite kortet

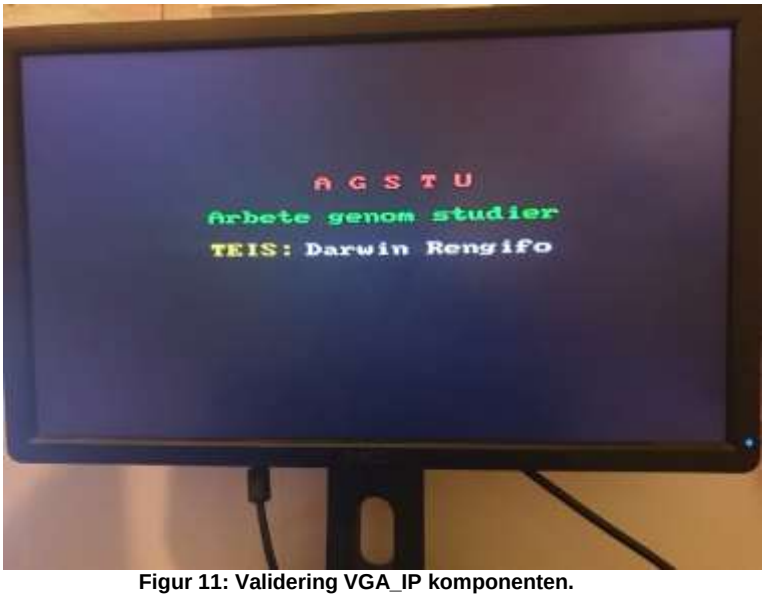
6. VALIDERING KONSTRUKTIONEN

6.1 Testprotokoll för validering

I Tabell 8 visar testprotokoll för validering och figur 11 visar validering i VGA-skärmen.

Tabell 8. Testprotokoll för validering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	Röd färg	- Visar texten ”AGSTU” på skärmen	Ja(se figur 11)
2	Grön färg	- Visar texten ”Arbete genom studier” på skärmen	Ja(se figur 11)
3	Gul färg	- Visar texten ”TEIS” på skärmen	Ja(se figur 11)
4	Vit färg	- Visar texten ”Darwin Rengifo” på skärmen	Ja(se figur 11)



Figur 11: Validering VGA_IP komponenten.

7. MJUKVARAN ARKITEKTUR

7.1 Minneskarta

Memory-map från ”sumary.html” för konstruktionen, se figur 12.

Nios II Memory Map

Slave Descriptor	Address Range	Size	Attributes
jtag_uart	0x08089038 - 0x0808903F	8	printable
sysid	0x08089030 - 0x08089037	8	
TIMER_HW_IP_0	0x08089020 - 0x0808902F	16	
pll_0	0x08089010 - 0x0808901F	16	
pio_in_key	0x08089000 - 0x0808900F	16	
onchip_ram	0x08084000 - 0x08087FFF	16384	memory
VGA_HW_IP_0	0x08000000 - 0x0807FFFF	524288	
new_sdram_controller_0	0x04000000 - 0x07FFFFFF	67108864	memory

Figur 12. Flödesdiagram

7.2 Drivrutiner

Det visar en översikt över de funktioner som krävdes av kravspecifikationen.

7.2.1 VGA färger

Värde (hexadecimalt)	Färg
0x00	
0x01	
0x02	
0x03	
0x04	
0x05	
0x06	
0x07	

7.2.2 VGA-drivrutiner

Tabell 9: ”DE10_Lite_VGA_Driver.h” innehåller funktionsdeklarationer och inkluderas för att få tillgång till vga-rutinerna.

Tabell 10: ***”draw_vga.c”*** innehåller vga-rutinerna.

Tabell 11: ***”DE10_Lite_VGA_Driver.h”*** innehåller också definitioner av macron som fungerar för att läser och skriver värden i minnesplacering.

Tabell 9. Funktionsdeklarationer i DE10_Lite_VGA_Driver.h

Funktion	Beskrivning
void clear_screen(alt_u8 rgb);	Rensar skärmen med färgen rgb.
void print_str(alt_u32 x, alt_u32 y, alt_u8 rgb, alt_u8 bg_rgb, alt_u8 *str);	Skriver ut en sträng på koordinaterna x,y med färgen rgb och bakgrundsfärgen bg_rgb. Skulle strängen fortsätta utanför högerkanten fortsätter den på raden under. Skulle den gå utanför nederkanten fortsätter den längst upp till vänster.

Tabell 10. VGA-rutinerna i draw_vga.c

Funktion	Beskrivning
clear_screen(BLACK);	skriver över alla adresser i bild minnet med färgen black=0x00.
print_str(135,80,"A G S T U",RED,0);	skriver ut texten ”TEIS” i röd=0x04 färg på mitten skärmen.
print_str(90,100,"Arbete genom studier",GREEN,0);	skriver ut texten ”Arbete genom studier” i grön=0x02 färg på under första texten på skärmen.
print_str(90,120,"TEIS:",YELLOW,0);	skriver ut texten ”TEIS” i gul=0x06 färg på under andra texten på skärmen.
print_str(135,120,"Darwin Rengifo",WHITE,0);	skriver ut texten ”Darwin Rengifo” i vit=0x07 färg på under tredje texten på skärmen.

Tabell 11. VGA-macron i DE10_Lite_VGA_Driver.h

Funktion	Beskrivning
#define write_pixel(x,y,color) IOWR_32DIRECT(**VGA_HW_IP_0_BASE**,((320*(y)+(x))*4),(color))	Skriver värde ”color” till position x_pos,y_pos i minnet.
#define read_pixel(x,y) IORD_32DIRECT(**VGA_HW_IP_0_BASE**,(320*(y)+(x))*4)	Läser värde från x_pos, y_pos i minnet.

7.3 C koden

I figur 13 visar c-koden för att validerar VGA_IP komponent på mjukvaran.

```
#include <stdio.h>
#include <alt_types.h>
#include <draw_vga.h>           //device driver for skrivs

#define BLACK       0x00
#define BLUE        0x01
#define GREEN       0x02
#define CYAN        0x03
#define RED         0x04
#define MAGENTA     0x05
#define YELLOW      0x06
#define WHITE       0x07

int main()
{
    clear_screen(BLACK);
    while(1){
        print_str(135,80,"A G S T U",RED,0);
        print_str(90,100,"Arbete genom studier",GREEN,0);
        print_str(90,120,"TEIS:",YELLOW,0);
        print_str(135,120,"Darwin Rengifo",WHITE,0);
    }
  return 0;
}
```

Figur 13. VGA_IP validering